PROPUESTA DE ACTA DE REUNIÓN

Alumno:

Hugo Pérez Muñoz

Asignatura y Profesor:

Proyecto Intermodular. Francisco José Mera Calderón

ACTA DE REUNIÓN

1. Introducción

El presente documento describe la propuesta del Proyecto Intermodular para el ciclo formativo de Desarrollo de Aplicaciones Web (DAW).

El objetivo es el análisis, diseño e implementación de "ChronoWork", un sistema integral para el registro de la jornada laboral. En un contexto donde la normativa exige un control horario preciso, esta aplicación web busca automatizar y securizar dicho proceso.

El sistema permitirá a los empleados realizar fichajes, gestionar ausencias y consultar su cómputo de horas. A su vez, RRHH podrá gestionar empleados y generar informes, garantizando la integridad de los datos mediante un sistema de roles que incluye un Administrador y un Inspector con permisos de auditoría

2. Justificación (Por qué)

En el contexto laboral actual, el registro de la jornada es una obligación legal en España. Muchas pymes aún dependen de sistemas manuales (Excel, hojas de firmas) que son ineficientes, propensos a errores y fácilmente manipulables. Esto genera una notable inseguridad jurídica para la empresa y el empleado.

ChronoWork nace para solucionar este problema real, ofreciendo una herramienta digital, centralizada y segura. El proyecto busca optimizar la gestión de RRHH, reducir la carga administrativa y, fundamentalmente, garantizar la transparencia e inviolabilidad del registro horario.

3. Objetivos (Para qué)

Objetivo General: Desarrollar una aplicación web integral para la gestión del registro de jornada que cumpla con la normativa laboral , optimizando el control horario para RRHH y garantizando la integridad de los datos.

Objetivos Específicos:

- Implementar un sistema de fichaje (Clock-in/Clock-out) intuitivo.
- Desarrollar un dashboard para que el empleado consulte su cómputo de horas.

- Crear un sistema de roles (Empleado, Administrador, Inspector) con permisos definidos.
- Implementar un módulo CRUD para la gestión de empleados por parte del Administrador.
- Diseñar un flujo de "Solicitud y Aprobación" para correcciones y ausencias.
- Asegurar que los administradores no puedan modificar registros unilateralmente.
- Generar informes exportables (PDF/Excel) para auditorías.
- Garantizar la seguridad mediante autenticación robusta y comunicaciones cifradas.

4. Alcance del Proyecto (Quién lo usará)

El proyecto está dirigido a PYMEs que necesiten digitalizar su registro horario. Los usuarios se dividen en tres roles principales:

- **Empleados:** Realizan fichajes y gestionan sus solicitudes.
- **Administradores (RRHH):** Gestionan empleados, contratos y aprueban solicitudes.
- **Inspectores:** Rol de solo lectura para auditorías y garantizar la transparencia.

El proyecto SÍ incluye:

- El desarrollo del backend (API REST) y frontend (SPA).
- La implementación de los tres roles mencionados.
- El sistema de autenticación y login.
- Módulo de fichaje (Clock-in / Clock-out).
- Módulo CRUD de gestión de empleados.
- Sistema de solicitudes y aprobaciones.
- Generación de informes básicos en PDF/Excel.

NO se incluyen en esta fase:

- El desarrollo de una aplicación móvil nativa (aunque la web será responsive).
- La integración directa con software de terceros (ERPs o sistemas de nóminas).
- Funcionalidades avanzadas de RRHH (evaluación de desempeño, turnos complejos).

5. Metodología y Tecnologías (Cómo se hará)

La metodología de trabajo será ágil, inspirada en **Scrum**. El proyecto se dividirá en Sprints (ciclos de 2-3 semanas) con objetivos definidos. Se utilizará **GitHub** para el control de versiones y la gestión de tareas.

El sistema se basa en una arquitectura de cliente-servidor desacoplada:

- **Frontend (Cliente):** Next.js (Framework de React) con TypeScript.
- **Backend (Servidor):** NestJS (Framework de Node.js) con TypeScript.
- **Base de Datos:** PostgreSQL.
- **Plataforma BBDD:** Supabase (Backend-as-a-Service).
- **Entorno de desarrollo:** Visual Studio Code, Postman, Git.

6. Planificación Temporal

El proyecto se desarrollará en una duración total estimada de 18 semanas , divididas en 5 fases principales:

1. **Fase 1. Análisis (2 semanas):** Definición de requisitos, casos de uso y selección de la pila tecnológica.
2. **Fase 2. Diseño (3 semanas):** Diseño de arquitectura, Modelo E-R de la BBDD, diseño de API y wireframes (UI).
3. **Fase 3. Desarrollo (8 semanas):** Implementación de Backend (NestJS), BBDD (Supabase/PostgreSQL) y Frontend (Next.js). Integración de API.
4. **Fase 4. Pruebas (3 semanas):** Pruebas unitarias, de endpoints, funcionales y de integración. Corrección de errores.
5. **Fase 5. Despliegue y Documentación (2 semanas):** Configuración del entorno de producción, despliegue y redacción de manuales.

7. Requisitos del Sistema

Requisitos Funcionales:

- **RF-Autenticación:** Gestión de roles (Empleado, Administrador, Inspector) con permisos diferenciados.
- **RF-Fichaje:** Sistema de Clock-in y Clock-out.
- **RF-Gestión:** Módulo CRUD para empleados y tipos de contrato.

- **RF-Workflows:** Sistema de solicitud y aprobación para correcciones y ausencias.
- **RF-Informes:** Generación de reportes de horas exportables (PDF/Excel).
- **RF-Auditoría:** Registro de logs de modificaciones (visible para el Inspector).

Requisitos No Funcionales:

- **RNF-Seguridad:** Encriptación de contraseñas y uso obligatorio de HTTPS.
- **RNF-Integridad:** Garantizar que los fichajes no puedan ser modificados unilateralmente por un Administrador.
- **RNF-Rendimiento:** Tiempo de respuesta para operaciones críticas < 3 segundos.
- **RNF-Usabilidad:** Interfaz intuitiva, clara y *responsive* (adaptable a móviles).

Requisitos No Funcionales:

- **Funcionamiento en navegadores modernos**

8. Recursos Necesarios

- **Hardware:** Ordenador personal con entorno de desarrollo
- **Software y Servicios:** IDE (Visual Studio Code), Node.js, Git, Supabase (para PostgreSQL), Postman y navegadores web.
- **Otros:** Conexión a Internet, repositorio en GitHub y acceso a documentación técnica.

9. Riesgos y Limitaciones

Riesgos:

- **Complejidad de la Integridad:** Asegurar la inviolabilidad de los datos (evitar que un Admin modifique fichajes) es un reto técnico central.
- **Curva de aprendizaje:** La pila tecnológica es moderna (NestJS, Next.js) y su integración desacoplada requiere una planificación de API cuidadosa.
- **Gestión del tiempo:** Las 8 semanas de desarrollo son ajustadas para la cantidad de funcionalidades (CRUD, workflows, informes).

Limitaciones (explícitas del alcance):

- La aplicación será *responsive*, pero no se desarrollará una app móvil nativa.
- No se contempla la integración con software externo de nóminas o ERPs.
- No se incluyen funcionalidades avanzadas de RRHH (gestión de turnos, evaluación de desempeño).

10. Resultados Esperados

Al finalizar el proyecto, se entregarán los siguientes elementos:

- **Código fuente completo** del frontend y backend.
- **Base de datos implementada** (esquema o script).
- **Manual de usuario**.
- **Manual técnico** (de la arquitectura y despliegue).
- **Informe final del proyecto** (este documento ampliado).
- **Presentación de defensa**

11. Conclusión

El desarrollo de "ChronoWork" permitirá a las empresas modernizar y digitalizar un proceso crítico y obligatorio como es el registro de la jornada. Este proyecto reemplazará los sistemas manuales, ineficientes y poco fiables, por una herramienta automatizada, segura e íntegra.

La aplicación resultante optimizará la gestión de RRHH y mejorará la transparencia y confianza con los empleados , resultando en una herramienta integral y eficiente que aplica las competencias adquiridas en el ciclo DAW.